



AUDIT REPORT

May 2026

For



Table of Content

Executive Summary	03
Number of Security Issues per Severity	04
Summary of Issues	05
Checked Vulnerabilities	06
Techniques and Methods	07
Types of Severity	09
Types of Issues	10
Severity Matrix	11
 Informational Issues	12
1. close_orders Cross-Namespace Dedup Silently Skips Legitimate Closures	12
2. Missing Event Emission in Beneficiary Management Functions	13
3. Unchecked Arithmetic in attest_order	14
4. Missing validate_address on Role Addresses in initialize	15
5. Instruction order_id Not Cross-Referenced With order_tracker.order_id in replenish	16
Closing Summary & Disclaimer	18



Executive Summary

Project Name	Zynk Labs
Project URL	https://www.zynk.money/
Overview	The Zynk Protocol is a Solana-based smart contract that facilitates token transfers between different parties through a managed operator system. It implements a sophisticated architecture with timelock mechanisms, signature verification, and order tracking capabilities.
Audit Scope	The scope of this Audit was to analyze the Zynk Labs Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Zynklabs/zynk-protocol-solana-programs/tree/zynk-core-v4/programs/zynk-core
Branch	zynk-core-v4
Contracts in Scope	lib.rs
Commit Hash	451c58005a4610e99c21f60e2dd54ae007434d0f
Language	Rust
Blockchain	Solana
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	6th May 2026 - 9th May 2026
Updated Code Received	11th May 2026
Review 2	11th May 2026 - 12th May 2026
Fixed In	77256f02e667fd41d7690402b622cfa02f9f8202

Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	0 (0.0%)
Low	0 (0.0%)
Informational	5 (100.0%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	0	0
	Partially Resolved	0	0	0	0	0
	Resolved	0	0	0	0	5



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	close_orders Cross-Namespace Dedup Silently Skips Legitimate Closures	Informational	Resolved
2	Missing Event Emission in Beneficiary Management Functions	Informational	Resolved
3	Unchecked Arithmetic in attest_order	Informational	Resolved
4	Missing validate_address on Role Addresses in initialize	Informational	Resolved
5	Instruction order_id Not Cross-Referenced With order_tracker.order_id in replenish	Informational	Resolved



Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:



Signer authorization



Account data matching



Sysvar address checking



Owner checks



Type cosplay



Initialization



Arbitrary cpi



Duplicate mutable accounts



Bump seed canonicalization



PDA Sharing



Incorrect closing accounts



Missing rent exemption checks



Arithmetic overflows/underflows



Numerical precision errors



Solana account confusions



Casting truncation



Insufficient SPL token account verification



Signed invocation of unverified programs



Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.



Severity Matrix

		Impact		
		 High	 Medium	 Low
Likelihood	 High	Critical	High	Medium
	 Medium	High	Medium	Low
	 Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Informational Issues

close_orders Cross-Namespace Dedup Silently Skips Legitimate Closures

Resolved

Path

lib.rs

Description

The close_orders function iterates over remaining_accounts and deduplicates by order_id at line 818:

```
let order_id = order_tracker.order_id;
if order_ids.contains(&order_id) { continue; }
order_ids.push(order_id);
```

Two distinct OrderTracker PDAs can share the same order_id value while existing in different PDA namespaces:

- Create/replenish path: {ORDER_TRACKER_SEED, partner_id, order_id}
- Attest path: {ORDER_TRACKER_SEED, b"attest", order_id}

If both are passed as remaining_accounts, the first is closed and the second is silently skipped by the contains check. The admin cannot close both in a single close_orders invocation.

Impact

Operational inconvenience only. The admin must call close_orders in two separate transactions to close both namespace variants of the same order_id. No fund loss or security impact.

Recommendation

Deduplicate by account address instead of order_id to allow closing different-namespace trackers with the same order_id in a single call.

Note

Deduplication on account address implemented.

Commit hash: 7d2cea49411ec4c40dcdbdcd482cd30d1a5aadca



Missing Event Emission in Beneficiary Management Functions

Resolved

Path

lib.rs

Description

The three beneficiary management functions perform critical state changes to the beneficiary whitelist without emitting any Anchor events. By contrast, every other state-modifying function in the protocol emits events (OrderCreated, OrderReplenished, OrdersClosed, OrderAttested, Action for all timelock lifecycle transitions).

- `whitelist_beneficiary` creates a new Beneficiary PDA with no event.
- `toggle_beneficiary` flips the `is_active` flag with no event.
- `revoke_beneficiary` permanently closes a Beneficiary PDA with no event.

Impact

Off-chain monitoring systems have no event-based mechanism to detect beneficiary whitelist changes in real time. If a compromised admin or guardian modifies the whitelist, there is no event-based forensic trail. Detection relies on polling account state or parsing transaction instructions, which is less reliable and introduces latency.

Recommendation

Define and emit events for all three operations:

```
#[event]
pub struct BeneficiaryWhitelisted {
    pub partner_id: [u8; 32],
    pub public_key: Pubkey,
    pub allow_transient: bool,
    pub authority: Pubkey,
}

#[event]
pub struct BeneficiaryToggled {
    pub partner_id: [u8; 32],
    pub public_key: Pubkey,
    pub is_active: bool,
    pub authority: Pubkey,
}

#[event]
pub struct BeneficiaryRevoked {
    pub partner_id: [u8; 32],
    pub public_key: Pubkey,
    pub authority: Pubkey,
}
```

Emit in each respective function body. For `revoke_beneficiary`, the event must be emitted inside the function body before the `Anchor close` directive executes in the instruction epilogue.

Note

Events added.

Commit hash: 7da51dc8baeacdfaaba91e46d05f95993cb683a3



Unchecked Arithmetic in attest_order

Resolved

Path

lib.rs

Description

The settlement check in attest_order at line 768 uses unchecked addition:

```
require!(  
    order_tracker.amount_in + amount >= order_tracker.amount_out,  
    CustomError::DeficientOrder  
);
```

The + operator on u64 can silently wrap around on overflow. In contrast, the same pattern in replenish (L674-676) correctly uses checked_add:

```
order_tracker.amount_in = order_tracker.amount_in  
    .checked_add(amount)  
    .ok_or(ProgramError::ArithmeticOverflow)?;
```

Impact

Theoretical. Exploiting this requires amount_in to be near u64::MAX, which is impractical given real token supplies. However, keeping defense coding practice it should be consistent with the rest of the program.

Recommendation

Use checked_add consistently:

```
require!(  
    order_tracker.amount_in  
        .checked_add(amount)  
        .ok_or(ProgramError::ArithmeticOverflow)?  
    >= order_tracker.amount_out,  
    CustomError::DeficientOrder  
);
```

Note

Checked addition applied.

Commit hash: c8f9b2361f76652105ae6fd0a447738b56b3f413



Missing validate_address on Role Addresses in initialize

Resolved

Path

lib.rs

Description

The initialize function validates token mint addresses via `validate_address` (L376) to reject the null pubkey, but does not apply the same validation to the three role addresses set from instruction arguments:

```
config.admin = admin;           // L370 - not validated
config.guardian = guardian;     // L371 - not validated
config.attester = attester;     // L372 - not validated
```

If any of these is set to `Pubkey::default()` (the 32-byte zero address), the corresponding role's functions become permanently inaccessible because no account can sign as the zero address.

Impact

Misconfiguration during initialization could permanently lock critical protocol functions. Since the Config PDA is a singleton created via `init`, re-initialization is impossible without program redeployment.

- **Zero admin:** locks `execute_request`, `unpause`, `revoke_timelock`, `close_orders`.
- **Zero guardian:** locks `ack_timelock`, `execute_consensus`, all beneficiary management.
- **Zero attester:** locks attestation signatures, consensus co-signing, `pause` (for attester role).

Recommendation

Perform the validation in the initialize function

```
validate_address(&admin)?;
validate_address(&guardian)?;
validate_address(&attester)?;
```

Note

Validations added.

Commit hash: `e0a48384b39b8c72b957162c3f36d1d678bb3d9d`



Instruction order_id Not Cross-Referenced With order_tracker.order_id in replenish

Resolved

Path

lib.rs

Description

The replenish function accepts order_id: {u8; 32} as an instruction argument (L642). However, this argument is never validated against the on-chain order_tracker.order_id. The only place the instruction order_id is used is in the emitted OrderReplenished event (L691):

```
emit!(OrderReplenished {  
    order_id, // ← instruction arg, NOT  
    order_tracker.order_id  
    zynk_op_vault: order_tracker.zynk_op_vault,  
    // ...  
});
```

Meanwhile, the Replenish account struct derives the OrderTracker PDA using the on-chain stored values (L1269):

```
seeds = [ORDER_TRACKER_SEED, order_tracker.partner_id.as_ref(),  
order_tracker.order_id.as_ref()],
```

The #[instruction(order_id: {u8; 32})] annotation at L1253 makes the instruction arg available to the struct, but it is not used in any seed derivation or constraint. The PDA is resolved entirely from self-referential on-chain data.

This means the manager can pass any arbitrary order_id in the instruction, and it will be emitted in the event without validation – while the actual order being replenished is determined solely by the OrderTracker account passed in the transaction.

Impact

Off-chain indexers and monitoring systems that rely on the OrderReplenished event's order_id field to correlate replenishments with orders will receive incorrect data if the manager passes a mismatched order_id. This does not affect on-chain state or fund flows – the correct OrderTracker is still modified – but it corrupts the off-chain audit trail.

A malicious or buggy off-chain caller could emit events that associate replenishments with the wrong orders, making reconciliation and accounting unreliable.

Recommendation

Either validate the instruction order_id matches the on-chain value:

```
require!(order_id == order_tracker.order_id, CustomError::InvalidOrder);
```



Or remove the instruction argument entirely and use the on-chain value in the event:

```
pub fn replenish(
  ctx: Context<Replenish>,
  // order_id removed from params
  amount: u64,
  close_order: bool,
  meta: Option<Vec<EventArg>>
) -> Result<()> {
  // ...
  emit!(OrderReplenished {
    order_id: order_tracker.order_id,    // use on-chain value
    // ...
  });
}
```

Note

Removed `order\_id` from params and used on-chain value for `order\_id`.

Commit hash: 77256f02e667fd41d7690402b622cfa02f9f8202



Closing Summary

In this report, we have considered the security of Zynk Labs Contract. We performed our audit according to the procedure described above.

5 issues of informational severity were found and the Zynk Labs Team resolved all the issues.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

May 2026

For



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com